

Preview

Claude full tutorial

1. What Is Claude Code?

Claude Code is an AI coding agent that can read files, run commands, edit code, and verify changes.

* Agent workflow: Gather context → Plan → Act → Verify → Repeat.

* Terminal CLI: Full agent workflow.

* VS Code: Work with code and visual diffs inside the editor.

* Desktop app: Use Claude Code without working directly in the terminal.

✓ Claude Code operates on tasks rather than only answering questions.

★ Tokens affect both usage cost and how much information Claude can retain in its context window.

2. Daily-Use Slash Commands

Slash commands help you manage tasks, sessions, context, and settings.

* ``/help`` - List available commands.

* ``/init`` - Draft a `CLAUDE.md` for the project.

* ``/clear`` - Clear the conversation and start fresh.

* ``/compact`` - Summarize the conversation while retaining context.

* ``/context`` - Check context usage.

* ``/rewind`` - Return to an earlier point.

- * ``/memory`` - View and edit memory files.
- * ``/usage`` - Check plan usage.
- * ``/rename`` - Rename the current session.
- * ``/resume`` - Reopen an earlier session.
- * ``/btw`` - Ask a side question outside the main conversation history.
- * ``/effort`` - Adjust how much effort Claude applies.
- * ``/model`` - Switch models.
- * ``Esc`` - Stop Claude while it is working.
- * ``Esc Esc`` - Open the rewind flow.

💡 Use ``/context`` regularly to understand what is filling Claude's context window.

★ Use ``/clear`` when switching to an unrelated task.

3. Modes - How Claude Works

Modes control how Claude approaches tasks and handles permissions.

- * **Manual Mode:** Default mode. Asks before edits and commands.
- * **Accept Edits Mode:** Allows file edits without asking each time; other risky actions may still require approval.
- * **Plan Mode:** Explores the code and creates a plan without making changes.
- * **Auto Mode:** Uses a safety checker to approve actions considered safe.
- * **Bypass Permissions Mode:** Runs without permission prompts. Use only inside a throwaway container or VM.

✓ Plan Mode is useful when a task involves multiple files or requires analysis before implementation.

💡 Switch modes using `Shift + Tab`.

★ Modes control how Claude operates; models control which AI model Claude uses.

4. Models and Cost

Models differ in capability and cost. The prices below are from the provided notes.

* Fable - \$10 input / \$50 output per million tokens: Hardest reasoning and coding tasks.

* Opus - \$4 input / \$20 output per million tokens: Complex changes and major decisions.

* Sonnet - \$2 input / \$10 output per million tokens: Everyday coding.

* Haiku - \$1 input / \$5 output per million tokens: Small, simple, repeated tasks.

* Fast Mode: Same Opus model, faster responses, double the price; Opus only.

✓ Match the model to the complexity of the task and the cost of a mistake.

★ Prices are reproduced from your notes.

5. The `.claude` Folder

The `.claude` folder stores project-specific instructions, reusable workflows, agents, rules, and settings.

* `CLAUDE.md` - Project instructions Claude reads at the start of a session.

* `CLAUDE.local.md` - Private project-specific notes that are not shared.

- * `~/claude/CLAUDE.md` - Personal instructions used across projects.
- * `.claude/skills/` - Reusable workflows loaded when needed.
- * `.claude/agents/` - Specialist agents with their own instructions and tools.
- * `.claude/rules/` - Rules that apply to specific folders.
- * `.claude/settings.json` - Project settings, including hooks.

What Goes in `CLAUDE.md` ?

- * Project facts and important context.
- * Rules Claude must follow.
- * Warnings and details Claude cannot infer.
- * Avoid vague advice and information Claude can discover by reading files.

✓ `CLAUDE.md` helps avoid repeating project instructions in every conversation.

💡 Use `/init` to create a starting draft, then review and remove anything unnecessary.

★ Keep it concise because its contents consume context every session.

6. Skills, Hooks, Subagents and Plugins

Skills

A skill is a reusable playbook Claude loads when needed.

- * Stored in `.claude/skills/`.
- * Used for repeatable, task-specific workflows.
- * Can be invoked directly or selected by Claude when the request matches.

Hooks

Hooks run commands automatically at specific events.

- * Configured in ``.claude/settings.json``.
- * Can run checks such as linting or type-checking.
- * Can also create audit logs or notifications.

★ Review hook commands before enabling them because they run automatically.

Subagents

Subagents are separate Claude instances with their own context.

- * Explore: Fast, read-only investigation.
- * Plan: Read-only planning.
- * General-purpose: Can investigate and act.
- * Custom subagents can be created for tasks such as code review.

✓ Subagents help keep lengthy research out of the main conversation.

Plugins

Plugins bundle reusable capabilities.

- * Can include skills, hooks, agents, and MCP servers.
- * Help package capabilities for reuse.

Which One Should You Use?

- * Always-needed instructions → ``.CLAUDE.md``
- * Reusable task-specific workflow → Skill
- * Action that must run automatically → Hook

* *Separate investigation* → *Subagent*

* *Bundle of capabilities* → *Plugin*

7. MCP vs CLI vs API

These provide different ways for Claude or software to interact with tools and services.

* *MCP*: An open standard that connects Claude to external tools such as GitHub, Notion, Slack, browsers, and databases.

* *CLI*: A command-line tool that can be run directly or used by Claude.

* *API*: Allows software to communicate programmatically with another service.

✓ The notes recommend using a CLI when a suitable command-line tool already exists, and MCP when there is no suitable CLI.

💡 Disconnect MCP servers you do not use to reduce unnecessary startup and context overhead.

★ MCP provides tools the agent can use; a CLI is a command-line interface; an API enables programmatic communication.

8. Recommended Workflow

Follow the sequence: *Explore* → *Plan* → *Code* → *Verify* → *Commit*.

* *Explore*: Ask Claude to understand the relevant files and existing implementation.

* *Plan*: Use Plan Mode for complex changes or tasks involving multiple files.

* *Code*: Review and approve the plan before implementation.

* *Verify*: Run the relevant tests, build, or lint checks.

* *Commit*: Review the changes before committing.

Prompting Tips

- * State the goal and expected outcome.
- * Specify the expected output, such as a file, function, or code change.
- * Provide relevant context and constraints.
- * Explain why a result is wrong and what the correct result should achieve.
- * Ask what changed, why it changed, how it was verified, and what risks remain.

✓ Give Claude a clear pass/fail verification signal.

★ Do not treat a task as complete just because Claude says it is done.

9. Context Management and Recovery

Context contains the conversation, files read, command outputs, instructions, and tool information.

- * `/context`` - Check context usage.
- * `/compact`` - Summarize the current conversation.
- * `/clear`` - Start a fresh conversation.
- * Subagents - Delegate research that would otherwise consume substantial context.
- * `/btw`` - Ask a side question without adding it to the main history.

Recovery

- * Claude is going in the wrong direction → Press ``Esc``.
- * One small edit went wrong → Use ``Esc Esc``.
- * Several changes went wrong → Use ``/rewind``.

* Claude keeps repeating the same mistake → Use ``/clear`` and restart with better context.

* Switching to a new task → Use ``/clear``.

💡 The notes suggest considering compaction as context usage increases, and starting fresh when context becomes too crowded.

★ ``/compact`` keeps the task going with a summary; ``/clear`` resets the conversation.

10. Automation

Claude Code can run one-off tasks or repeated routines.

* One-shot task: Give Claude an instruction to complete and return a result.

* Scheduled or event-based routine: Run repeated tasks, such as checking GitHub issues and generating a summary.

* Cost control: Use smaller models for simple tasks, clear context between unrelated tasks, and disconnect unused MCP servers.

✓ Automation is useful for tasks that follow a repeatable process.

★ Review the output of automated tasks, especially when they can make changes.

11. Quick Reference

* Understand a codebase → Ask Claude to explain its structure and flow.

* Make a complex change → Use Plan Mode first.

* Store project instructions → ``CLAUDE.md``.

* Reuse a workflow → Skill.

* Run an automatic check → Hook.

- * *Delegate research → Subagent.*
 - * *Connect external tools → MCP or CLI.*
 - * *Context is getting full → `/context`, then `/compact`.*
 - * *Start a new task → `/clear`.*
 - * *Finish a coding task → Verify with a test, build, or lint check.*
-  *Start with a task you already repeat regularly, then decide whether it needs a skill, hook, or subagent.*
-  *Keep instructions focused, review changes, and verify results before considering the task complete.*